

SALESTORM

SYSTEM DESIGN

1. INTRODUCTION

SALESTORM is a high-scale e-commerce flash-sale system designed to handle a very large number of simultaneous purchase requests for a limited quantity of products.

The primary challenge is a scenario where 10,000 customers attempt to purchase a product at nearly the same time, while only 100 units are available.

The system is designed to maintain inventory correctness under high concurrency while providing a smooth purchasing experience.

2. REQUIREMENT ANALYSIS

SALESTORM is a high-scale e-commerce flash-sale system designed for a situation where thousands of customers compete for very limited inventory. The core challenge is to handle 10,000 simultaneous purchase attempts for 100 available units while guaranteeing zero overselling, safe reservations, idempotent payment processing and reliable order creation.

2.1 Business Objectives

- Handle sudden flash-sale traffic spikes without system collapse.
- Prevent inventory overselling under concurrent requests.
- Provide temporary inventory reservation with expiry and release.
- Ensure safe and idempotent payment processing.
- Maintain a consistent and traceable order lifecycle.
- Recover gracefully from payment, service and downstream failures.
- Support horizontal scaling and operational monitoring.

2.2 Actors

- Customer
- Admin
- Payment Gateway
- Shipment/Delivery Partner
- Notification Provider

2.3 Critical Business Scenario

10,000 customers → 100 available units → maximum successful reservations = 100 → overselling = 0

3. FUNCTIONAL REQUIREMENTS

3.1 User Management

- Registration/login
- Authentication and authorization
- Profile management

3.2 Product Management

- View/search products
- Product details, price and sale information
- Stock status

3.3 Flash Sale

- Configure/start/end sale
- Display sale price
- Accept controlled purchase requests

3.4 Virtual Queue

- Join queue
- Generate queue token
- Maintain queue position
- Control admission

3.5 Inventory

- Check stock
- Reserve stock atomically
- Confirm stock
- Release stock
- Track available/reserved/sold quantities

3.6 Reservation

- Create reservation
- Expiry timer
- Release expired reservation
- Confirm reservation
- Idempotent reservation

3.7 Cart and Checkout

- Add/update/remove cart items
- Validate reservation
- Calculate amount
- Start checkout

3.8 Payment

- Initiate payment
- Handle SUCCESS/FAILED/TIMEOUT
- Idempotent payment
- Reconciliation

3.9 Order

- Create order
- Confirm/cancel order
- Track order state

3.10 Shipment

- Create shipment
- Tracking number
- Shipment status

3.11 Notification

- Payment, order, shipment and expiry notifications

3.12 Admin

- Monitor queue, inventory, reservations, payments, orders and system health

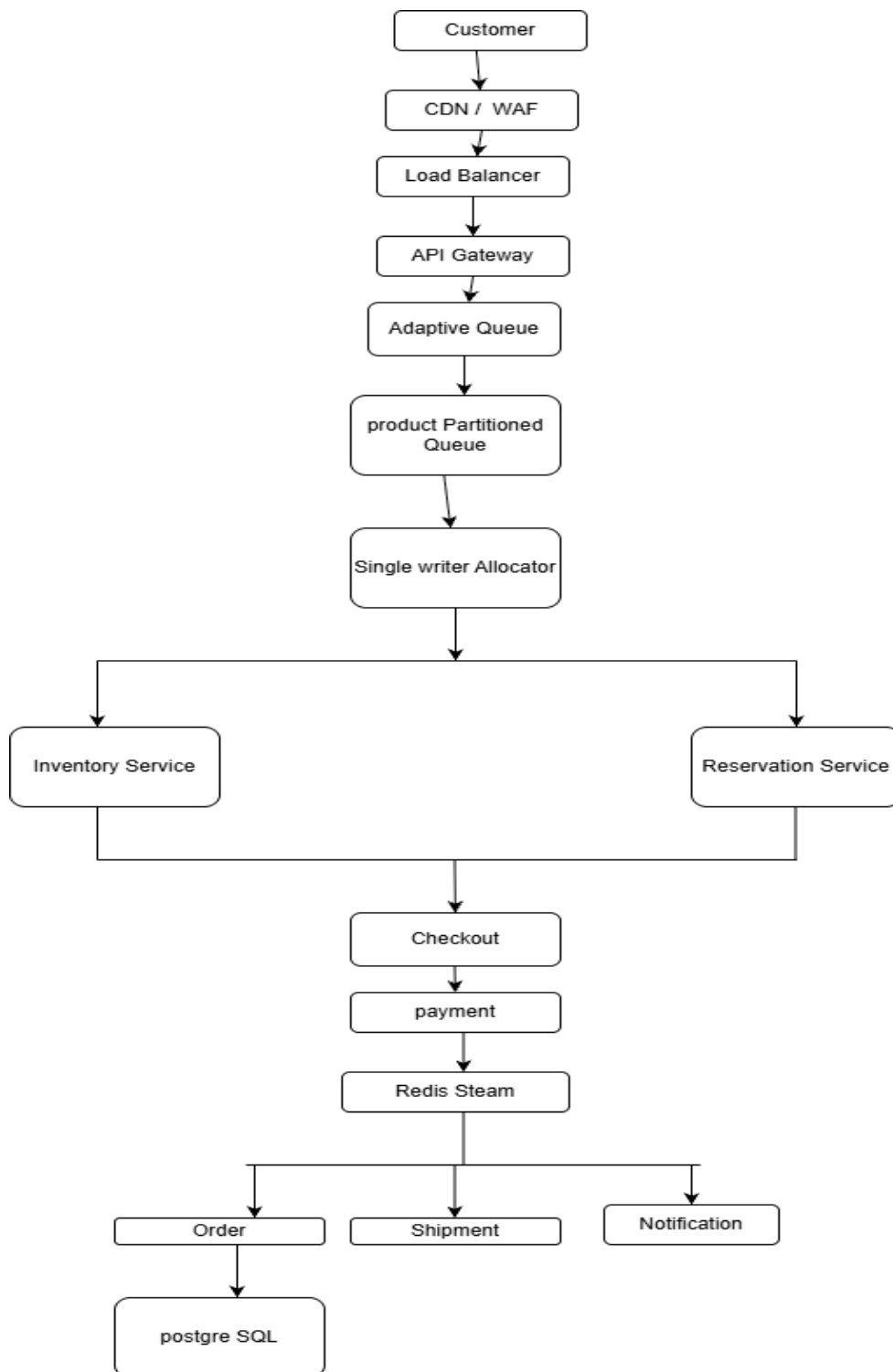
3.13 Failure Recovery

- Retry/replay recoverable events
- Dead-letter handling
- Recover from temporary downstream failures

4. NON-FUNCTIONAL REQUIREMENTS

normal Quality Attribute	SALESTORM Requirement
Scalability	Handle 10,000 concurrent requests through controlled admission and horizontally scalable services.
Performance	Keep API latency low and prevent flash-sale bursts from overwhelming transactional systems.
Consistency	Inventory must never become negative; successful reservations must never exceed available stock.
Availability	Temporary failure of non-critical downstream services should not lose successful business events.
Reliability	Use idempotency, retries, reconciliation and event replay.
Security	Authentication, authorization, HTTPS, validation, rate limiting, secrets protection and audit logs.
Maintainability	Modular services, SOLID principles, patterns and documented APIs.
Observability	Logs, metrics, traces, correlation IDs and dashboards.
Recoverability	Reservation expiry, payment reconciliation and event retry/replay.
Auditability	Trace important reservation, payment and order transitions.

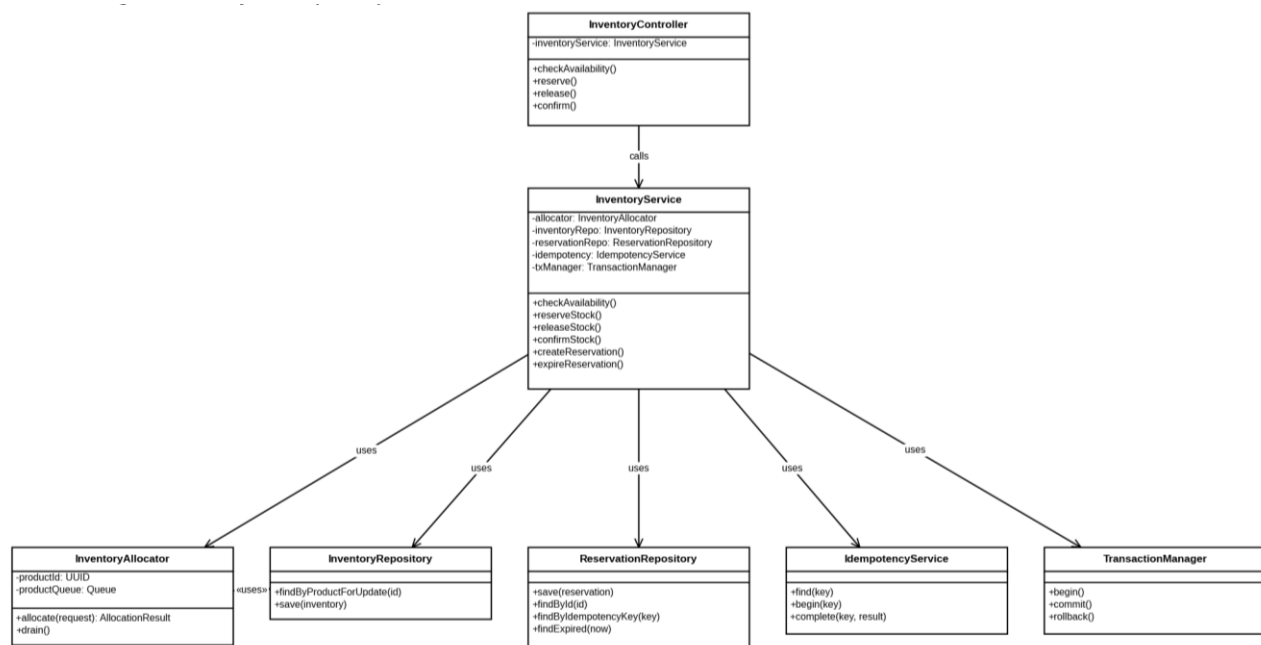
5.ARCHITECTURE



6. LOW LEVEL DESIGN

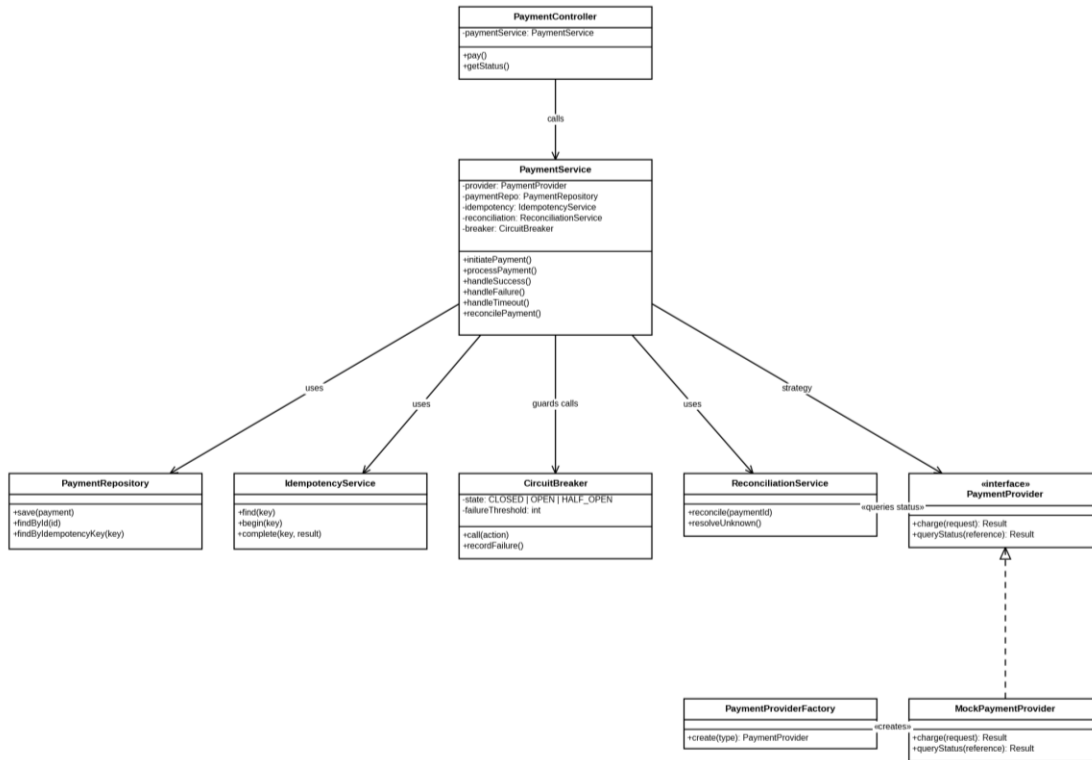
6.1 Inventory Module

The Inventory Module manages stock availability, reservation, release, and confirmation. It uses the **Single-Writer Inventory Allocator** to prevent overselling during flash sales. Transactions and idempotency ensure safe and consistent inventory updates. It supports the principle “**Parallel across products, serialized within a product.**”



6.2 Payment Module

The Payment Module handles payment initiation, success, failure, and timeout scenarios. It uses **Strategy**, **Factory**, **Circuit Breaker**, and **Idempotency** patterns. The Reconciliation Service resolves uncertain or timed-out payments. A Mock Payment Provider is used for prototype payment processing.



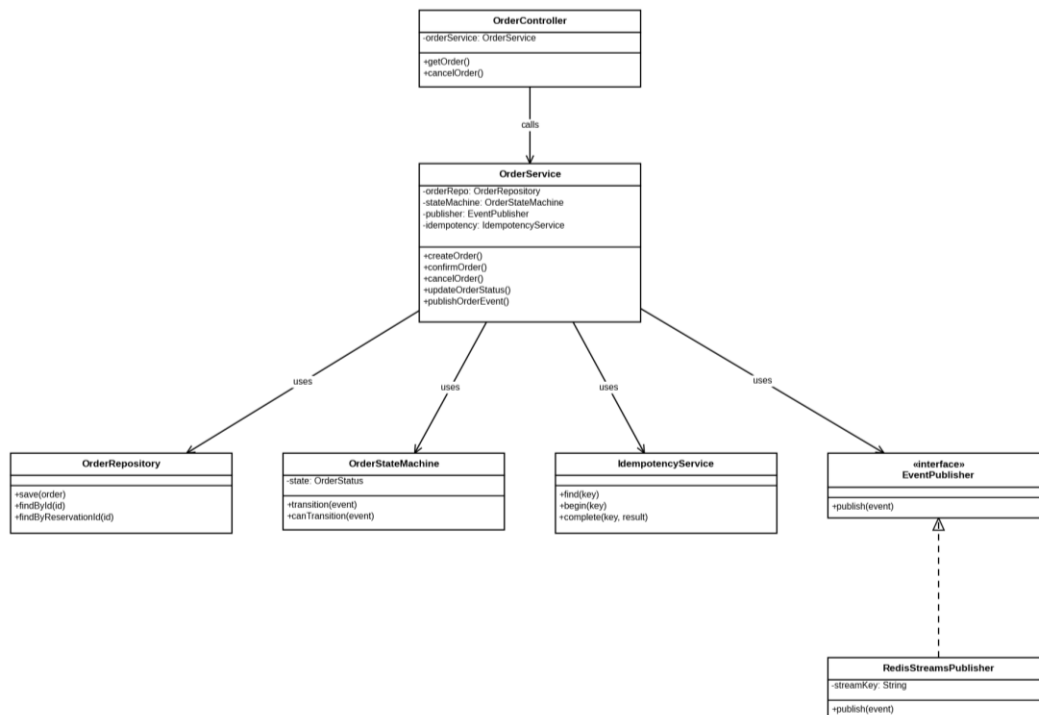
6.3 Order Module

The Order Module manages order creation, confirmation, cancellation, and status updates.

It uses a **State Machine** to control valid order transitions.

The Event Publisher with Redis Streams enables event-driven communication.

Idempotency prevents duplicate order creation.

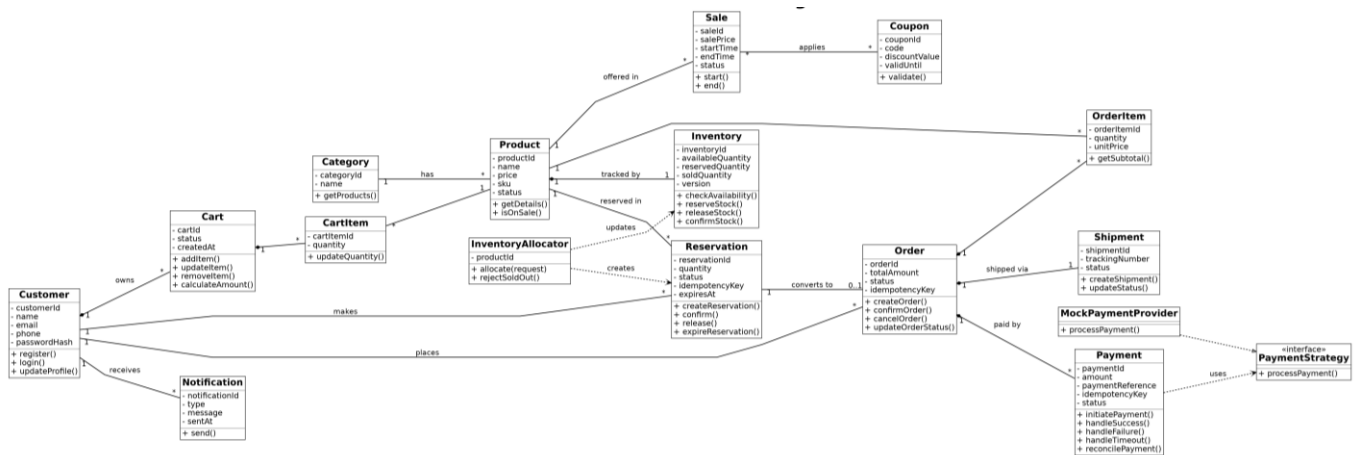


The Customer entity manages user information and is connected to carts, orders, reservations, payments, and notifications. Product and Category manage the product catalog, while Inventory tracks available, reserved, and sold quantities.

During a purchase, a customer can add products to a Cart, create a Reservation, and eventually generate an Order. Each order can contain multiple Order Items, have a corresponding Payment, and be fulfilled through Shipment.

The database also supports flash-sale functionality through the Sale, Coupon, and Sale_Coupon entities. Unique constraints and idempotency keys help prevent duplicate reservations, orders, and payments while maintaining data consistency during high traffic

8. UML DIAGRAM



The UML Class Diagram represents the main classes and their interactions in the SALESTORM e-commerce system.

It covers Customer, Product, Cart, Inventory, Reservation, Order, Payment, Shipment, Sale, Coupon, and Notification modules.

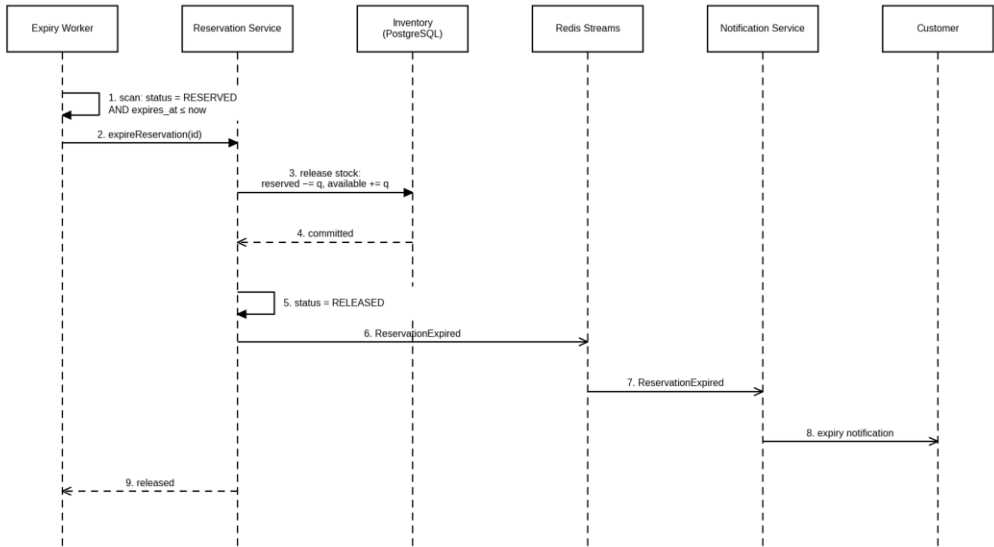
The diagram shows class attributes, methods, relationships, and dependencies between different components.

It also highlights the InventoryAllocator for controlled stock allocation and PaymentStrategy for flexible payment processing.

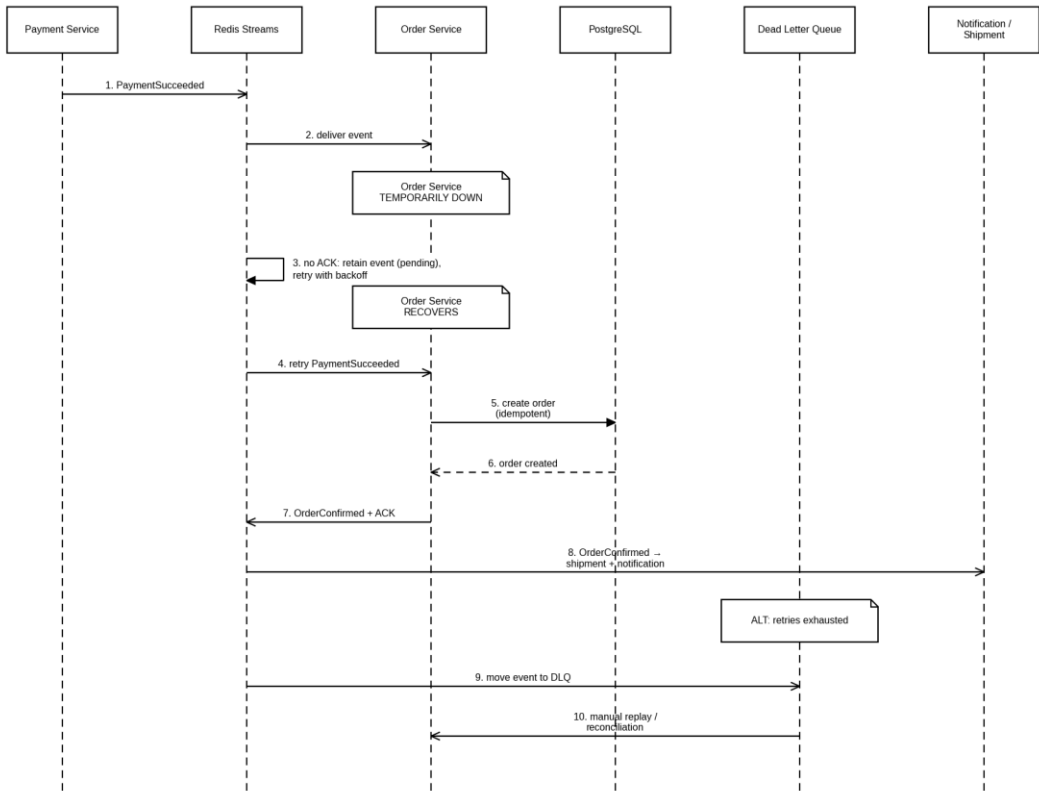
Together, these classes support the complete workflow from product selection and reservation to payment, order processing, shipment, and notification.

9. SEQUENCE DIAGRAM

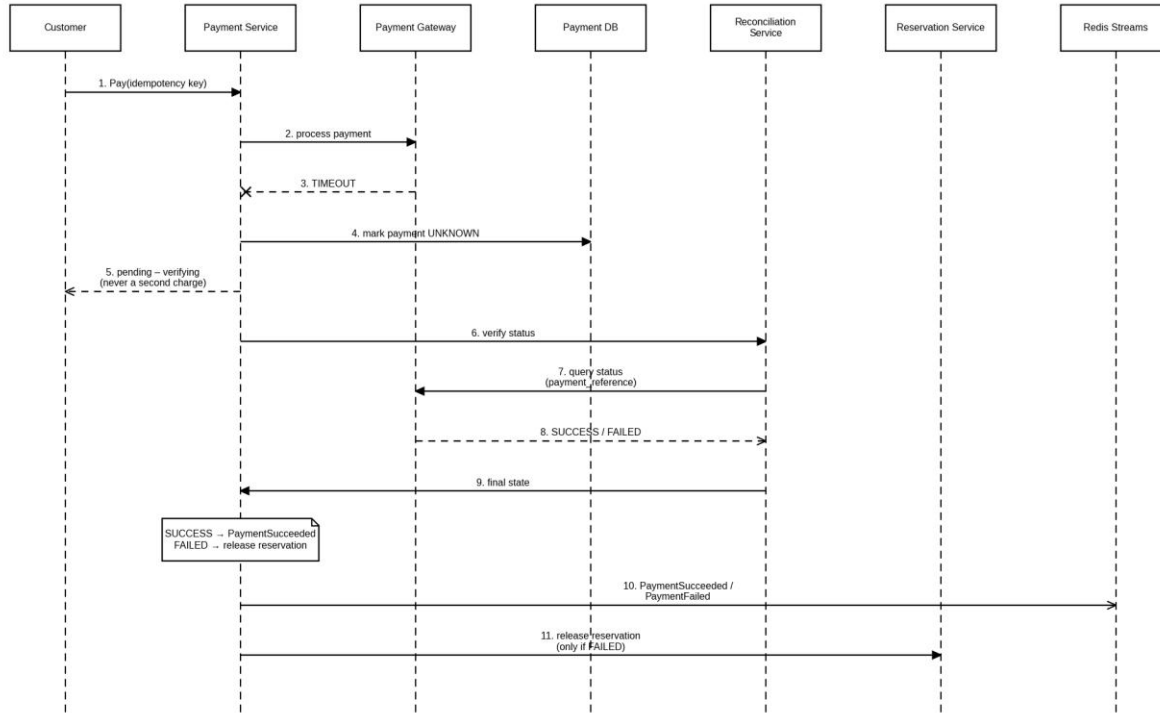
Sequence Diagram – Reservation Expiry & Stock Release



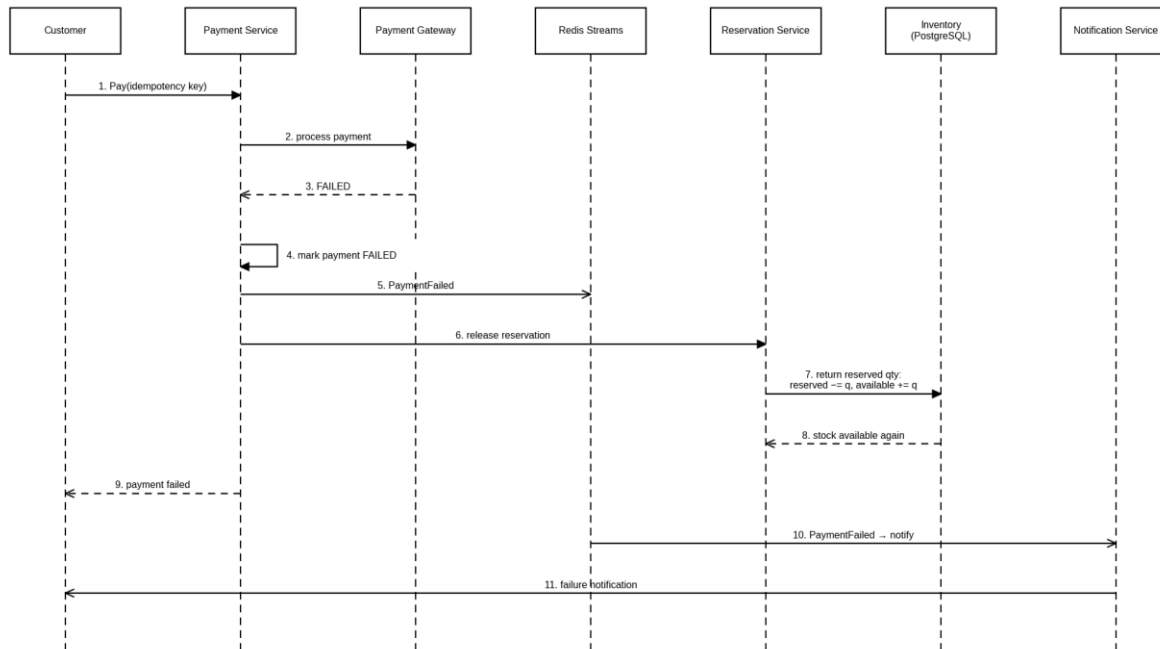
Sequence Diagram – Order Service Failure (event retry & replay)



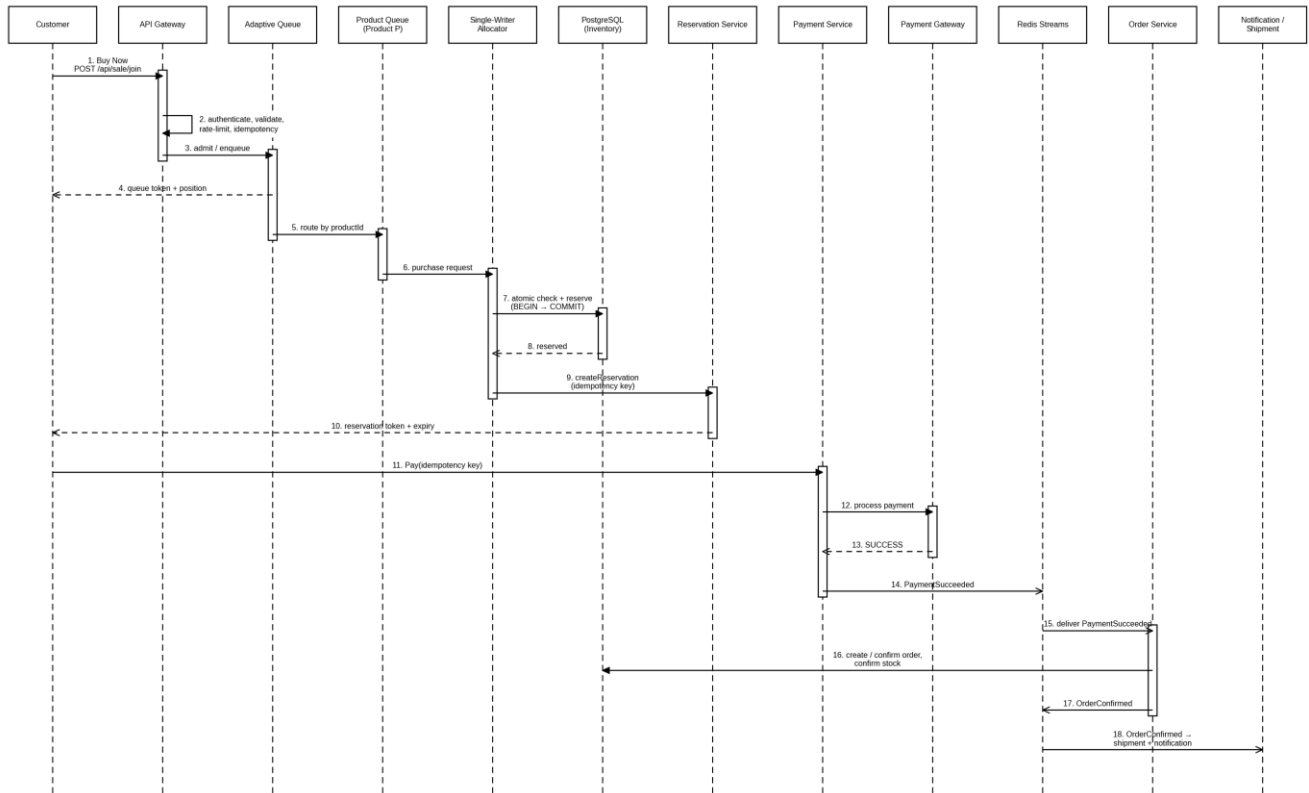
Sequence Diagram – Payment Timeout & Reconciliation



Sequence Diagram – Payment Failure



Sequence Diagram – Successful Purchase



10. API DESIGN

Method	Endpoint	Purpose
GET	/api/products	List products
GET	/api/products/{id}	Product details
POST	/api/sale/join	Join flash-sale queue
GET	/api/sale/queue/{token}	Queue status
POST	/api/reservations	Create reservation
GET	/api/reservations/{id}	Reservation status
DELETE	/api/reservations/{id}	Release/cancel reservation
POST	/api/checkout	Start checkout
POST	/api/payments	Process payment
GET	/api/payments/{id}	Payment status
GET	/api/orders/{id}	Order details
GET	/api/admin/dashboard	Admin dashboard

11. SOLID Principles

- Single Responsibility: each service/class has one primary responsibility.
- Open/Closed: payment providers and strategies can be extended without changing core business logic.
- Liskov Substitution: provider implementations follow a common interface.
- Interface Segregation: use small focused interfaces such as PaymentProvider and EventPublisher.
- Dependency Inversion: business services depend on abstractions/repositories rather than infrastructure.

12. DESIGN PATTERNS

- Strategy – payment provider selection.
- Factory – creation of provider implementations.
- State – reservation and order state transitions.
- Adapter – external payment/shipping API integration.
- Repository – database access abstraction.
- Observer/Event-driven – downstream reaction to business events.
- Facade – simplified checkout orchestration.
- Circuit Breaker – protection from unhealthy external dependencies.

13. SCALABILITY

- Horizontal scaling of stateless API and business-service instances.
- Adaptive queue absorbs flash-sale bursts.
- Product partitions isolate hot products.
- Single-writer allocation avoids a global inventory lock.
- Different products process in parallel.
- Redis cache reduces repeated read load.
- Read replicas may support read-heavy product/catalog workloads.
- Event consumers scale independently.
- Hot products can receive dedicated queue/worker capacity.

14. RELIABILITY AND FAILURE RECOVERY

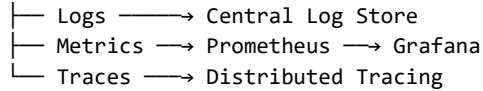
Failure	Recovery
Payment failure	Release reservation and return stock.
Payment timeout	Mark UNKNOWN and reconcile; never blindly create a second payment.
Duplicate request	Use idempotency key and return existing result.
Reservation expiry	Expiry worker releases stock.
Order service down	Keep PaymentSucceeded event and retry after recovery.
Event consumer failure	Retry with backoff and dead-letter handling.
External dependency outage	Timeout + circuit breaker.
Database contention	Queue critical inventory operations and use atomic transaction.

15. SECURITY

- HTTPS
- Authentication and role-based authorization
- API rate limiting
- Input validation
- Password hashing
- Secrets management
- Audit logging
- No real payment credentials in prototype
- Correlation IDs and secure operational logging

16. OBSERVABILITY

Applications



- Requests/sec
- Queue length
- p95/p99 latency
- Reservation success/failure
- Payment success/failure
- Order rate
- Database latency
- Event backlog
- Error rate
- Inventory inconsistency / overselling count

17. STRESS-TEST SCENARIO

Test configuration:

Concurrent users = 10,000
Initial inventory = 100
Purchase quantity = 1
Payment outcomes = SUCCESS / FAILED / TIMEOUT
Duplicate requests = enabled
Order Service outage = 30 seconds

Expected:

Successful reservations <= 100
Overselling = 0
Expired/failed stock = released
Duplicate transactions = 0
Payment success events = eventually converted to orders
Order outage = recovered through event retry/replay

Recommended prototype validation: Locust for concurrent load, Pytest for inventory concurrency/idempotency, Docker Compose for local infrastructure, and Prometheus/Grafana for metrics.